

Lab 4

Advanced Generic Systems & Exception Safety

1. Problem Overview

You are tasked with programming a **robotic warehouse controller**. The warehouse stores **Boxes** of various data types (integers, characters, or floats). The controller must track physical constraints such as *total weight* and *robot energy*.

Critical Requirement: Atomicity: If a bulk operation fails midway (e.g. the robot runs out of energy while moving multiple boxes), the system must *roll back* to its original state so that no inventory is lost or misplaced.

2. Structural Requirements

2.1. The `Box<T>` Template

Create a simple templated structure to represent a warehouse item:

- `T data`: the actual value being stored.
- `int weight`: the physical weight of the box.

2.2. The Storage<T> Class

Implement a templated stack as the raw storage medium. The `pop` operation must return the entire `Box` structure so the `Warehouse` can track weight correctly.

- `void push(T item, int weight)`: creates a `Box` and pushes it.
- `Box<T> pop()`: removes and returns the top `Box`; throws an exception if empty.
- `int currentSize()`: returns the number of items currently stored.

2.3. The Warehouse<T> Class

This class manages the logic and physical constraints of the system.

Attributes: `maxCapacity` (item count limit), `maxWeight` (total weight limit), `currentEnergy`, `currentWeight` (running total weight of all items currently stored).

Operation Logic & Precedence:

Dispatch: Single or Bulk

Step 1. Check Energy First. Ensure $energy \geq 2$. If not, throw `PowerFailure`.

Step 2. Check Inventory Second. Ensure storage is not empty. If empty, throw `OutofStock`.

Step 3. Execute. Decrement energy by 2, pop the item, and update total weight.

- **Receive:** `void receive(T data, int weight)`: Zero energy cost. Must verify `Capacity` first, then `MaxWeight`, before pushing. If both limits are violated, `WarehouseFull` takes precedence over `Overloaded`.
- **Recharge:** Adds to `currentEnergy`. There is *no* maximum energy cap. Produces no output.

3. Exception Specifications

Define a custom exception hierarchy with the triggering conditions and output strings shown below.

Exception Class	Triggering Condition	Output
PowerFailure	Dispatch attempted when <i>energy</i> < 2	ERR_POWER
WarehouseFull	Item received when <i>size</i> = <i>maxCapacity</i>	ERR_FULL
Overloaded	Receiving item would exceed <i>maxWeight</i> limit	ERR_WEIGHT
OutofStock	Dispatch attempted on empty storage	ERR_EMPTY

4. The Challenge: Atomic Bulk Dispatch

Implement `void bulkDispatch(int k)`, which attempts to dispatch k items in a single atomic operation.

The function removes items one by one, applying the same energy and stock checks as a regular dispatch. During this process, **no output is produced**; removed items must be kept track of so they can be restored if the operation fails.

If any removal fails (either a `PowerFailure` or `OutofStock`), the operation must be **fully rolled back**: all buffered items are pushed back in reverse order, energy is restored to what it was before the call, and the exception is re-thrown to the caller. The warehouse must end up in exactly the state it was in before `bulkDispatch` was called.

If all k removals succeed, print each item's data on a new line in the order they were removed.

Note: You may assume $k \geq 1$ for all `bulkDispatch` calls.

5. Input / Output Format

5.1. Input Specification

Line 1: Warehouse configuration (four space-separated values):

- **Capacity** (`int`): maximum item count.
- **MaxWeight** (`int`): maximum total weight.
- **InitialEnergy** (`int`): starting energy units.
- **Type** (`char`): item data type: `I` (int), `C` (char), `F` (float).

Line 2: integer N : total number of operations.

Subsequent lines: one operation per line:

Command	Meaning
1 <data> <weight>	Receive item (push to storage)
2	Dispatch item (pop and print data)
3 <value>	Recharge energy
4 <k>	Atomic bulk dispatch of k items

5.2. Sample 1

Input

```
2 100 3 I
4
1 10 60
1 20 50
2
4 2
```

Expected Output

```
ERR_WEIGHT
10
ERR_POWER
```

Explanation:

Command	What happens	State after
1 10 60	Success. Item received.	Wt:60 En:3 Sz:1
1 20 50	Fails. $60 + 50 = 110 > 100$. Throws Overloaded.	ERR_WEIGHT
2	Success. Item 10 popped; energy $3 \rightarrow 1$.	Wt:0 En:1 Sz:0
4 2	Fails. Energy $1 < 2$; PowerFailure on first item. No rollback needed.	ERR_POWER

5.3. Sample 2: Rollback Test

This test case specifically exercises the **atomic rollback** guarantee of `bulkDispatch`. The bulk operation partially succeeds before hitting a `PowerFailure`, at which point the stack and energy must be fully restored.

Input

```
4 200 6 I
7
1 10 40
1 20 50
1 30 30
4 4
1 99 60
4 2
2
```

Expected Output

```
ERR_POWER
99
30
20
```

Explanation: Refer to next page

Command	What happens	State after
1 10 40	Success. Stack: [10].	Wt:40 En:6 Sz:1
1 20 50	Success. Stack: [20, 10].	Wt:90 En:6 Sz:2
1 30 30	Success. Stack: [30, 20, 10].	Wt:120 En:6 Sz:3
4 4	Save state: energy= 6. Pops 30 (en $6 \rightarrow 4$), 20 (en $4 \rightarrow 2$), 10 (en $2 \rightarrow 0$). 4th check: $0 < 2 \Rightarrow$ PowerFailure . Rollback : re-push 10, 20, 30; restore energy to 6. Stack: [30, 20, 10].	ERR_POWER
1 99 60	Success. $120 + 60 = 180 \leq 200$. Stack: [99, 30, 20, 10].	Wt:180 En:6 Sz:4
4 2	Success. Pops 99 (en $6 \rightarrow 4$), 30 (en $4 \rightarrow 2$). Prints 99, 30. Stack: [20, 10].	Wt:90 En:2 Sz:2
2	Success. Pops 20; energy $2 \rightarrow 0$. Prints 20. Stack: [10].	Wt:40 En:0 Sz:1

6. Submission Guidelines

- Implement `Storage` using dynamic arrays or linked nodes. **STL is not permitted.**
- All class templates and member functions must be fully defined.
- All custom exceptions must inherit from `std::exception`.
- Do not use `if` conditions to avoid throwing. **Violation will result in 0 marks.**
- `bulkDispatch` must use `try-catch` internally to intercept, rollback, and re-throw.
- Each operation in `main` must be wrapped in its own `try-catch` that catches `std::exception` and prints `e.what()`.
- Implement destructors to prevent memory leaks.